Mean Tardiness Still Favors the Searched Sequence

Muting (Don) Ma

The University of Alabama, Tuscaloosa, AL, USA

mma10@ua.edu

Abstract

A planner who sequences jobs on a flow line where jobs cannot pass one another must commit to one job order before the processing times are known. I compare the earliest due date (EDD) rule, a dispatching rule widely used in practice, with the iterated greedy algorithm of Ruiz and Stützle adapted to an eight-stage permutation flow shop with release dates and total weighted tardiness. On instances with 15, 45, and 120 jobs, iterated greedy reduces total weighted tardiness relative to EDD by 98.5, 97.6, and 81.0 percent, and a mixed-integer program solved with HiGHS proves its values optimal on the instances with 6 and 15 jobs. I then execute both fixed sequences on 180 paired scenarios of the 45-job instance in which each processing time is multiplied by an independent lognormal factor with mean one. At a coefficient of variation of 0.2, the iterated greedy sequence has the lower weighted tardiness in all 30 scenarios, with a mean of 420.6 weighted minutes against 3,541.1, an 88.1 percent reduction. As the coefficient of variation rises from 0.1 to 0.8, the relative reduction falls from 94.6 to 27.2 percent while the mean absolute reduction stays between 2,572.0 and 3,683.7 weighted minutes; at 0.8, the 10th percentile of the paired reduction falls to 851.5 weighted minutes and EDD is better in 1 of 30 scenarios. Processing-time variability therefore erodes the advantage of the searched sequence first in the worst periods, while its mean advantage persists across the range tested.

Keywords: Scheduling; Permutation flow shop; Weighted tardiness; Iterated greedy; Stochastic processing times

1. Introduction

On flow lines such as printed circuit board assembly or furniture finishing, every job visits the same stages in the same order, and when jobs cannot pass one another, the order chosen at the first stage holds at every stage. A job that is quick at the first stage and slow at a later one then delays every job behind it, and the resulting waiting accumulates downstream. The planner of such a line fixes one sequence for the jobs of a planning period before processing begins, although the jobs are released at different times, carry due dates of unequal importance, and take processing times that will differ from their planned values. Dispatching rules are used very often in practice, and the earliest due date (EDD) rule is among the most common rules for tardiness objectives [1]. EDD orders the jobs by due date alone, so it cannot account for job weights, release dates, or processing times at the stages after the first.

Minimizing total tardiness in a permutation flow shop is NP-hard in the ordinary sense with one machine and in the strong sense with two or more [1], and the iterated greedy algorithm of Ruiz and Stützle [2] was the best of the methods its authors tested for weighted tardiness. That evaluation, like the review of Vallada et al. [1], used instances with fixed processing times and no release dates. Studies of flow shops with random processing times have concentrated on the expected makespan, for which heuristics adapted from deterministic procedures, including sequences computed from mean processing times, perform close to the best stochastic methods [3–5], and a review of 100 studies of flow shops under uncertainty identifies tardiness and its weighted form as criteria that remain to be examined [6]. A

broader review of schedule execution concludes that optimized predictive schedules outperform simple rules when uncertainty is low and lose that advantage beyond a level of variability that depends on the system [7]. For weighted tardiness with release dates, it therefore remains open whether a searched sequence, held fixed, keeps its advantage over the EDD sequence as processing times vary, and at which level of variability that advantage weakens.

I adapt the iterated greedy algorithm of Ruiz and Stützle [2] to an eight-stage permutation flow shop with release dates and total weighted tardiness, and I compare it with EDD on four synthetic instances with 6 to 120 jobs. A mixed-integer program solved with HiGHS [8] certifies the optimal values of the two smallest instances, and a checker that shares no code with either method recomputes the feasibility and the measures of every schedule. I then execute both sequences, unchanged, on 180 paired scenarios of the 45-job instance in which each processing time is multiplied by an independent lognormal factor with mean one, at coefficients of variation from 0.1 to 0.8.

On exact data, iterated greedy reduces total weighted tardiness relative to EDD by 98.5, 97.6, and 81.0 percent on the instances with 15, 45, and 120 jobs, and it attains the proven optima on the instances with 6 and 15 jobs. Across the 180 paired scenarios, the iterated greedy sequence has the lower weighted tardiness in 179, and its mean reduction stays between 2,572.0 and 3,683.7 weighted minutes at every coefficient of variation tested. The relative reduction, however, falls from 94.6 percent at a coefficient of variation of 0.1 to 27.2 percent at 0.8, and at 0.8 the 10th percentile of the paired reduction falls to 851.5 weighted minutes.

The results extend the finding for the expected makespan, that sequences computed on mean processing times remain effective, to total weighted tardiness with release dates, and they locate the erosion that Aytug et al. [7] describe in the tail of the per-period gain before its mean. For a planner, the searched sequence is worth adopting at every level of variability I tested, and the variability of the line is the quantity that determines how dependably its gain is realized in a given period.

2. Related literature

2.1 Heuristics for tardiness in permutation flow shops settle the deterministic case

Vallada et al. [1] reviewed and evaluated 40 heuristics and metaheuristics for total tardiness in the m-machine permutation flow shop on a common benchmark. Among the heuristics, those built on inserting and interchanging jobs performed best, and EDD ranked below the variant of the insertion heuristic of Nawaz, Enscore, and Ham (NEH) that starts from the EDD order. Ruiz and Stützle [9] introduced iterated greedy for the makespan: it removes a few jobs from the incumbent sequence, reinserts each at its best position with the NEH insertion step, and accepts the result under a constant-temperature criterion, with four removed jobs as the most effective setting in their experiments. Ruiz and Stützle [2] extended the algorithm to flow shops with sequence-dependent setup times and the weighted tardiness objective, starting from an NEH variant that orders jobs by d_j/w_j; that start beat plain NEH on 470 of 480 instances, and the version with local search had the lowest average deviation from the best known solutions in all four instance sets.

All three studies evaluate sequences on instances with fixed processing times and without release dates. They therefore establish which sequence is best for the data as given, but not how a sequence chosen on those data performs when the realized times differ.

2.2 Studies of uncertain processing times settle the expected makespan

In a systematic review of 100 studies of flow shops under uncertainty published from 2001 to 2016, processing time was the uncertain parameter in 74 percent, and most studies optimized the makespan alone [6]. Baker and Altheimer [3] tested constructive heuristics adapted from deterministic procedures for the stochastic flow shop with lognormal processing times, on instances with up to 10 jobs and 6 machines, and found them within 1 percent of the best sequences found on average, against 5.3 percent for a random sequence. Framinan and Perez-Gonzalez [4] tested larger instances with coefficients of variation from 0.01 to 1.5 and concluded that the deterministic NEH applied to mean processing times performs extremely well for the expected makespan; following Hopp and Spearman, they classify coefficients of variation below 0.75 as low variability. Juan et al. [5] combined iterated local search with Monte Carlo simulation on the premise that good deterministic sequences remain good at moderate variability, and they advocated comparing the whole distribution of outcomes, beyond the mean.

Aytug et al. [7] classify the execution of schedules under uncertainty into completely reactive, robust, and predictive-reactive approaches, and they describe right-shifting, which keeps the sequence and delays operations as disruptions occur. Summarizing earlier simulation studies, they conclude that optimized predictive schedules outperform myopic dispatching when uncertainty is low, and that beyond a level of variability that depends on the system the advantage fades. Both Baker and Altheimer [3] and González-Neira et al. [6] name tardiness criteria as open directions. What remains unsettled is how a searched sequence compares with a rule-based sequence for total weighted tardiness with release dates when both are executed unchanged, and whether variability erodes the mean of that comparison or only its worst outcomes.

3. Problem formulation

3.1 A planner fixes one sequence for jobs with release dates, due dates, and weights

A line of m stages, indexed by k in line order, processes n jobs, indexed by j, and each stage has one machine. Job j is released at time r_j, is due at d_j, carries a weight w_j that scores its priority, and requires a processing time $p_{jk} > 0$ at stage k. Every job visits every stage in line order without interruption, jobs cannot pass one another, so one sequence of the jobs holds at every stage, and the buffers between stages are unlimited. I assume that every stage is idle at time zero, that the planner fixes the sequence before the first job starts and does not revise it, and that among sequences with equal weighted tardiness the planner prefers less waiting between stages.

3.2 The earliest-start recursion determines the schedule of a sequence

A sequence π lists the jobs in the order in which they enter the line, with $\pi(i)$ the job in position i. Given π, each operation starts at the later of the job's completion at the previous stage and the stage's completion of the preceding job:

$$S_{\pi(i),k} = \max\{C_{\pi(i),k-1},\ C_{\pi(i-1),k}\}, \qquad C_{\pi(i),k} = S_{\pi(i),k} + p_{\pi(i),k}, \tag{3.1}$$

with the conventions $C_{\pi(i),0} = r_{\pi(i)}$ and $C_{\pi(0),k} = -\infty$. The tardiness of job j is $T_j = \max\{0,\ C_{jm} - d_j\}$, and the primary and secondary measures are the total weighted tardiness and the total waiting between

stages,

$$Z(\pi) = \sum_{j=1}^{n} w_j T_j, \qquad W(\pi) = \sum_{j=1}^{n} \sum_{k=2}^{m} \big(S_{jk} - C_{j,k-1}\big), \tag{3.2}$$

in weighted minutes and minutes, respectively.

By induction over positions and stages, any feasible schedule that keeps sequence π starts each operation no earlier than (3.1), so the recursion gives every job its earliest completion at the last stage and π its smallest value of Z. The planning problem is therefore to choose the sequence that minimizes $Z(\pi)$, which is the permutation flow shop problem with release dates and total weighted tardiness. Applied with realized processing times, (3.1) is also the right-shift execution of a fixed sequence described by Aytug et al. [7], which I use in the stress test.

3.3 A positional mixed-integer program defines the optimum

To certify optimal values, I state the problem as a mixed-integer program over sequence positions $q = 1, \dots, n$. The binary variable x_{jq} equals 1 when job j occupies position q, B_{qk} and E_{qk} are the start and end of the job in position q at stage k, and $T_j \geq 0$ is the tardiness of job j:

$$\min \quad \sum_{j=1}^{n} w_j T_j \tag{3.3}$$

$$\text{s.t.} \quad \sum_{q=1}^{n} x_{jq} = 1 \quad \forall j, \qquad \sum_{j=1}^{n} x_{jq} = 1 \quad \forall q, \tag{3.4}$$

$$E_{qk} = B_{qk} + \sum_{j=1}^{n} p_{jk}\, x_{jq} \quad \forall q, k, \tag{3.5}$$

$$B_{q1} \geq \sum_{j=1}^{n} r_j\, x_{jq} \quad \forall q, \tag{3.6}$$

$$B_{qk} \geq E_{q,k-1} \quad \forall q,\ k \geq 2, \tag{3.7}$$

$$B_{qk} \geq E_{q-1,k} \quad \forall q \geq 2,\ k, \tag{3.8}$$

$$T_j \geq E_{qm} - d_j - U_j\,(1 - x_{jq}) \quad \forall j, q, \tag{3.9}$$

$$x_{jq} \in \{0, 1\}, \quad T_j \geq 0.$$

Constraints (3.4) place every job in exactly one position, (3.5) processes each operation without interruption, (3.6) and (3.7) enforce release dates and stage order, and (3.8) allows a stage to start a job only after it finishes the preceding position, which also keeps one order at every stage. Constraint (3.9) assigns the tardiness of the job in position q to that job and is inactive for every other job when $U_j = \max\{0,\ \max_h r_h + \sum_{h,k} p_{hk} - d_j\}$, where h ranges over all jobs.

Under (3.1), every job of every sequence finishes by $\max_h r_h + \sum_{h,k} p_{hk}$, because each completion time is a release date plus the processing times along one path through the schedule. The bound U_j therefore never forces a positive tardiness on the earliest-start schedule of any sequence, which by section 3.2 is the best schedule for that sequence, and the optimal value of the program equals $\min_\pi Z(\pi)$. I use the program only as a benchmark that certifies optimal values.

4. Solution methods

4.1 The earliest due date rule sorts the jobs once

The EDD rule sorts the jobs by d_j, breaks ties by job identifier, and times the resulting sequence π^{EDD} by (3.1). Its sequence ignores weights, release dates, and processing times, and it requires no search.

4.2 Iterated greedy searches the sequences by insertion

The method is IG_RS_LS, the iterated greedy algorithm with local search that Ruiz and Stützle [2] proposed for the flow shop with sequence-dependent setup times; the line studied here has no setup times, so I apply its weighted tardiness version with every setup time equal to zero, as shown in Algorithm 1. Sequences are compared by the pair $F(\pi) = (Z(\pi), W(\pi))$ in lexicographic order, and the best position for a job is the position in a partial sequence that yields the smallest F, the earliest on an exact tie. The initial sequence, NEH_EWDD, orders the jobs by nondecreasing d_j/w_j and inserts them one at a time at their best positions [2], following the insertion step of NEH as stated by Ruiz and Stützle [9]. Each iteration removes $\delta = 4$ jobs at random, reinserts them in the order of removal, and applies a local search that moves single jobs to their best positions until no move improves F. A worse sequence replaces the current one with probability $\exp(-\Delta/\Theta)$, where Δ is its increase in Z and $\Theta = \lambda \sum_{j,k} p_{jk}/(10\,n\,m)$ with $\lambda = 0.5$, which are the acceptance rule and the settings of Ruiz and Stützle [2].

Algorithm 1 Iterated greedy for weighted tardiness, after Ruiz and Stützle [2]. The pair $F = (Z, W)$ is compared lexicographically, and every insertion is made at the best position.

Require: instance; destruction size $\delta = 4$; temperature factor $\lambda = 0.5$; iteration limit $L = 5000$; seed 522
$\pi \leftarrow$ NEH_EWDD: jobs by nondecreasing d_j/w_j, each inserted at its best position
if $F(\pi^{\mathrm{EDD}}) < F(\pi)$ **then** $\pi \leftarrow \pi^{\mathrm{EDD}}$ ▷ never start worse than EDD
end if
$\pi \leftarrow$ LocalSearch(π); $\pi^{*} \leftarrow \pi$; $\Theta \leftarrow \lambda \sum_{j,k} p_{jk}/(10\,n\,m)$
for $\ell = 1, \ldots, L$ **do**
    remove δ jobs chosen at random from π, keeping the removed jobs in order of removal
    insert each removed job, in that order, at its best position
    $\pi' \leftarrow$ LocalSearch of the rebuilt sequence
    if $F(\pi') < F(\pi)$ **then** $\pi \leftarrow \pi'$; **if** $F(\pi) < F(\pi^{*})$ **then** $\pi^{*} \leftarrow \pi$
    else if $u \le \exp\bigl(-(Z(\pi') - Z(\pi))/\Theta\bigr)$ for u uniform on $[0, 1]$ **then** $\pi \leftarrow \pi'$
    end if
end for
return π^{*}

function LocalSearch(π)
    repeat
        for each job j, in random order **do**
            remove j and insert it at its best position, giving π''
            if $F(\pi'') < F(\pi)$ **then** $\pi \leftarrow \pi''$
            end if
        end for
    until a full pass changes nothing
    return π
end function

I break ties in Z by W, because the planner prefers less work in process among sequences with equal tardiness and because many sequences can share a value of Z when most jobs finish on time. I start from π^{EDD} whenever it improves on NEH_EWDD, so that the method never returns a sequence worse than

the baseline; on all four instances, NEH_EWDD was the better start. I stop after $L = 5000$ iterations with a fixed seed, where Ruiz and Stützle [2] stop at a CPU-time limit, so that a rerun on another computer reproduces each reported sequence. The best-position search times the jobs ahead of each candidate position once and abandons a position as soon as its partial pair (Z, W) can no longer improve on the best position found, which is exact because both totals are nondecreasing as jobs are appended.

5. Computational design

5.1 Four synthetic instances differ mainly in the length of the planning period

I use four synthetic instances generated for a graduate course in production scheduling, with all times in minutes, summarized in table 1. A 6-job instance on 4 stages is small enough to trace by hand and to solve by enumeration, and three development instances with 15, 45, and 120 jobs on 8 stages test the methods at increasing size. Processing times range from 2 to 20 minutes on every development instance, and a job's total processing time over the 8 stages averages between 85.1 and 88.8 minutes. Releases spread over 186, 597, and 1,687 minutes, which gives arrival rates between 0.071 and 0.081 jobs per minute, so the larger instances cover longer planning periods at a similar density. The median time from release to due date is 154 to 158 minutes, which leaves a median of 64.5 to 74 minutes that a job can spend waiting before it is late, and weights are integer priority scores from 1 to 5.

Table 1: The four instances. Processing times are per job and stage. The due-date window is the time from a job's release to its due date, and its median is taken over the jobs of the instance. All times are in minutes.

Instance	Jobs	Stages	Processing times	Releases	Median due-date window	Weights
Hand-checkable	6	4	3 to 19	8 to 78	30	1, 2, 4
Small development	15	8	2 to 20	24 to 210	158	1, 2, 4, 5
Medium development	45	8	2 to 20	10 to 607	154	1 to 5
Large development	120	8	2 to 20	3 to 1,690	154	1 to 5

5.2 Both methods run with fixed settings, and an independent checker scores every schedule

Iterated greedy runs with $L = 5000$ iterations and seed 522, with a safety cap of 300 seconds that no run reached, and I repeat it with seeds 1 to 5 to measure the dependence of the result on the seed. NEH_EWDD alone, the first step of Algorithm 1, is reported as a construction-only reference, so that the contribution of the search can be separated from that of the initial sequence. All runs use one laptop with an Intel Core Ultra 7 155U processor under Linux, Python 3.12, NumPy 2.5.4, and Numba 0.68.0, and computation time covers the method alone, after one warm-up run that compiles the timing code. I solve program (3.3) to (3.9) with HiGHS 1.15.1 [8] on the instances with 6 and 15 jobs, using one thread, a relative gap tolerance of zero, and a limit of 600 seconds; the larger instances are not attempted, because the 15-job instance already requires 85.8 seconds and more than 15,000 branch-and-bound nodes. I expect iterated greedy to attain the lower weighted tardiness on every instance, because it evaluates the tardiness of each candidate sequence, whereas EDD does not.

Each method's sequence, and the order read from each solution of the program, is timed by (3.1) and exported as the start and completion of every operation. A separately written checker reads only the instance data and the exported schedule, confirms every constraint of section 3.1 within a tolerance of

10^{-8} minutes, and recomputes Z, W, and the share of jobs finishing on time. Every value I report for a schedule is computed by this checker.

5.3 Thirty saved scenarios at each of six settings test the fixed sequences

In scenario ω, each processing time becomes $p_{jk}\,\varepsilon_{jk\omega}$, where the factors $\varepsilon_{jk\omega}$ are independent lognormal draws with mean one and coefficient of variation ν, obtained with $\sigma^2 = \ln(1+\nu^2)$ and a log-mean of $-\sigma^2/2$, the distribution used by Baker and Altheimer [3] and Framinan and Perez-Gonzalez [4]. For the main test I draw 30 scenarios of the 45-job instance at $\nu = 0.2$ from seed 20261103 and save them before either method runs, so that both sequences face identical realizations. Each sequence is the one computed on the exact data, it is executed unchanged in every scenario by the right-shift rule (3.1) with the realized processing times, and the checker confirms each realized schedule against the times of its scenario. For each sequence I report the mean and the 90th percentile, by linear interpolation, of Z and W over the scenarios, the number of scenarios in which a hard constraint is violated, and the mean, standard error, and 10th percentile of the paired reduction in Z.

I repeat the execution with 30 new scenarios at each of $\nu = 0.1$, 0.2, 0.3, 0.5, and 0.8, drawn from seeds 6522 to 6526, so that with the main test the study comprises 180 paired scenarios. This range covers the low-variability band below 0.75 in the classification used by Framinan and Perez-Gonzalez [4] and extends into the moderate band. I expect both sequences to become later as ν grows and the searched sequence to lose part of its advantage, because a sequence tuned to exact times has more on-time jobs that a delay can make late.

6. Results

6.1 Iterated greedy lowers weighted tardiness on every instance and reaches the proven optima

Table 2 reports every method on every instance, and every one of these schedules is feasible. Iterated greedy lowered total weighted tardiness relative to EDD by 17.1 percent on the 6-job instance, from 281 to 233 weighted minutes, and by 98.5, 97.6, and 81.0 percent on the development instances with 15, 45, and 120 jobs. It also lowered total waiting on the three development instances, by 65.6, 61.1, and 61.8 percent, but raised it on the 6-job instance from 13 to 37 minutes. Figure 1 shows both effects by instance size: the tardiness of the searched sequence is smallest relative to EDD at 15 and 45 jobs and rises to 19 percent of the EDD value at 120 jobs.

HiGHS proved the optimal total weighted tardiness to be 233 on the 6-job instance in 0.36 seconds and 21 on the 15-job instance in 85.8 seconds, and iterated greedy reached both values. On the 15-job instance, the solver's optimal order had 323 minutes of waiting and iterated greedy's had 252, because only iterated greedy breaks ties in Z by waiting. With seeds 1 to 5, iterated greedy returned exactly the default seed's tardiness on the 6-, 15-, and 45-job instances, and values from 4,159 to 4,306 on the 120-job instance, all below the default seed's 4,383. Its computation time grew from 0.21 seconds at 6 jobs to 37.2 seconds at 120 jobs, while EDD took under a millisecond on every instance.

6.2 The fixed searched sequence has the lower tardiness in all 30 scenarios at a coefficient of variation of 0.2

Table 3 summarizes the two fixed sequences for the 45-job instance over the 30 saved scenarios at $\nu = 0.2$. The iterated greedy sequence had the lower weighted tardiness in all 30 scenarios, with a

Table 2: Deterministic comparison. Each row is one method on one instance, and every schedule is feasible. Seconds are the method's computation time on the laptop of section 5.2. The optimum rows come from the mixed-integer program of section 3.3; on the 15-job instance the optimal tardiness of 21 was reached by an order with 323 minutes of waiting.

Instance (jobs)	Method	Total weighted tardiness	Total waiting	Seconds
Hand-checkable (6)	Earliest due date	281	13	0.000
	NEH_EWDD only	233	37	0.000
	Iterated greedy	233	37	0.21
	Optimum (HiGHS)	233	37	0.36
Small development (15)	Earliest due date	1,417	732	0.000
	NEH_EWDD only	70	291	0.000
	Iterated greedy	21	252	0.52
	Optimum (HiGHS)	21	323	85.8
Medium development (45)	Earliest due date	2,371	2,040	0.000
	NEH_EWDD only	360	1,068	0.000
	Iterated greedy	58	794	3.22
Large development (120)	Earliest due date	23,015	7,863	0.000
	NEH_EWDD only	9,452	3,928	0.003
	Iterated greedy	4,383	3,005	37.2

mean of 420.6 weighted minutes against 3,541.1, a reduction of 3,120.5 weighted minutes (standard error 150.1 over the paired scenarios) or 88.1 percent, and a 90th percentile of 592.2 against 4,616.6. In every scenario the reduction was at least 1,948.6 weighted minutes, and its 10th percentile was 2,235.1. Its mean waiting was 1,276.9 minutes against 2,454.5, a reduction of 48.0 percent. Neither sequence violated a hard constraint in any scenario, because right-shift execution preserves release dates, stage order, and one job per stage for any processing times, and due dates are soft targets. Figure 2 shows that the ranges do not overlap: the weighted tardiness of the iterated greedy sequence ranged from 103.3 to 1,120.0, and that of EDD from 2,276.2 to 6,013.8.

Table 3: Stress test on the 45-job instance: 30 scenarios at a coefficient of variation of 0.2, seed 20261103. "Planned" is the value of the sequence on the exact data; the mean and the 90th percentile are over the scenarios. Tardiness is in weighted minutes and waiting in minutes.

	Total weighted tardiness			Total waiting			Scenarios breaking
Sequence	Planned	Mean	90th pct.	Planned	Mean	90th pct.	a hard constraint
Earliest due date	2,371.0	3,541.1	4,616.6	2,040.0	2,454.5	2,768.6	0
Iterated greedy	58.0	420.6	592.2	794.0	1,276.9	1,454.9	0

6.3 The relative advantage narrows as variability grows, while the mean absolute advantage stays above 2,500 weighted minutes

Table 4 and fig. 3 repeat the execution at five coefficients of variation, each with 30 new scenarios. The mean weighted tardiness of the iterated greedy sequence rose from 148.1 at $\nu = 0.1$ to 8,901.6 at $\nu = 0.8$,

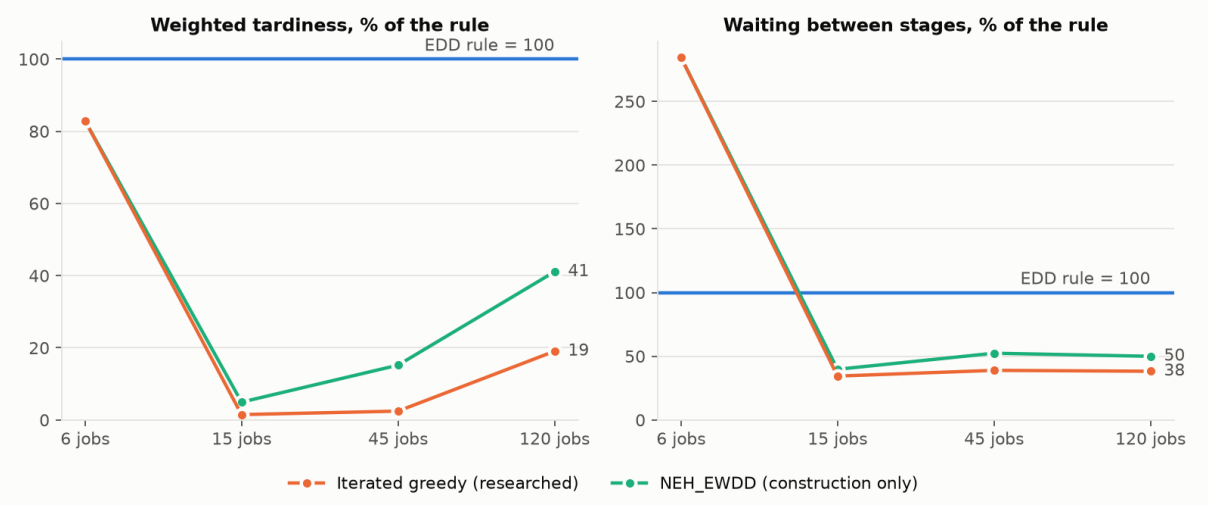


Figure 1: Total weighted tardiness (left) and total waiting (right) of iterated greedy and of NEH_EWDD alone, each as a percentage of the EDD value on the same instance (EDD is the line at 100). The horizontal axis gives the instance size; the labels at the right edge are the 120-job values. The two methods coincide on the 6-job instance.

and that of EDD from 2,720.1 to 12,230.5. The relative reduction in mean weighted tardiness fell from 94.6 percent at $\nu = 0.1$ to 27.2 percent at $\nu = 0.8$. The mean absolute reduction stayed between 2,572.0 and 3,683.7 weighted minutes at every level: it rose from $\nu = 0.1$ (standard error 65.9) to $\nu = 0.5$ (standard error 223.8), and its fall of 354.8 to 3,329.0 at $\nu = 0.8$ (standard error 342.7) is within the combined standard error of 409.3 of the two estimates. The gain in a single period became less dependable at $\nu = 0.8$: the 10th percentile of the per-scenario reduction, between 1,781.1 and 2,473.2 weighted minutes at the lower levels, fell to 851.5, and the EDD sequence won 1 of the 30 scenarios, by 870.4 weighted minutes, its only win in the 180 paired scenarios of the six sets.

Table 4: Sweep over variability on the 45-job instance. Each row uses 30 new scenarios, from seeds 6522 to 6526 in order. Tardiness is in weighted minutes. The reduction is the per-scenario difference between the two sequences' weighted tardiness: its mean, with the standard error in parentheses, and its 10th percentile; the percentage compares the two sequences' means.

	Earliest due date		Iterated greedy		Reduction		
ν	Mean	90th pct.	Mean	90th pct.	Mean (s.e.)	10th pct.	Percent
0.1	2,720.1	3,171.6	148.1	228.4	2,572.0 (65.9)	2,048.3	94.6
0.2	3,273.5	4,425.0	407.2	643.9	2,866.4 (121.3)	2,055.3	87.6
0.3	4,139.8	6,213.4	952.5	1,676.0	3,187.3 (203.8)	1,781.1	77.0
0.5	6,680.4	9,498.8	2,996.7	5,434.2	3,683.7 (223.8)	2,473.2	55.1
0.8	12,230.5	17,420.0	8,901.6	14,219.8	3,329.0 (342.7)	851.5	27.2

7. Discussion

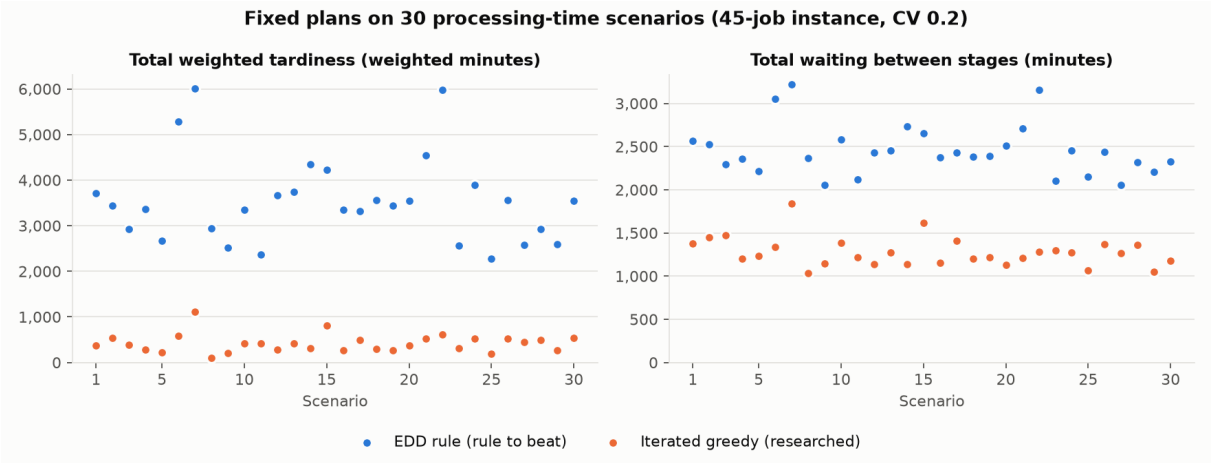


Figure 2: Total weighted tardiness (left) and total waiting (right) of the two fixed sequences in each of the 30 scenarios of table 3. Each dot is one sequence in one scenario, and both sequences face the same realized processing times in a scenario.

7.1 Weights and search explain the deterministic gain

Figure 4 draws both schedules of the 6-job instance. EDD places job J005, due at 67 with weight 1, ahead of jobs J003 and J004, which carry weight 4. Iterated greedy moves J005 to the end, so J003 finishes at 97 and J004 at 119, against 108 and 139 under EDD, while J005 finishes at 136, against 82. The weighted tardiness of J003 and J004 falls by 124 weighted minutes and that of J005 rises by 54, and with a smaller rise for J006 the total falls from 281 to 233. The same reordering raises waiting on this instance from 13 to 37 minutes, because the two jobs now at the end, J006 and J005, wait 18 and 19 minutes at stages S2 to S4 behind the long operations of the weight-4 jobs.

The share of jobs finishing on time rises from 0.333, 0.244, and 0.233 under EDD to 0.867, 0.911, and 0.642 under iterated greedy on the instances with 15, 45, and 120 jobs. NEH_EWDD alone already lowers weighted tardiness relative to EDD by 95.1, 84.8, and 58.9 percent on these instances (table 2), and the iterations of Algorithm 1 lower its value by a further 70.0, 83.9, and 53.6 percent, or 49, 302, and 5,069 weighted minutes. On the 120-job instance the search has not converged after 5000 iterations, since five other seeds find sequences 1.8 to 5.1 percent better than the default seed's 4,383, at a computation time that already reaches 37.2 seconds. On the 15-job instance, the optimal order found by the solver and the order found by iterated greedy share the tardiness of 21, but the solver's order has 323 minutes of waiting and iterated greedy's 252, so the tie-break removes 71 minutes of work in process that a tardiness-only objective leaves in place.

7.2 Variability turns the searched sequence's on-time jobs late

At $\nu = 0.2$, the iterated greedy sequence has 4 late jobs as planned and 11.5 on average across the 30 scenarios, while the EDD sequence has 34 and 36.1. The mean weighted tardiness of EDD rises by 1,170.1 weighted minutes over its planned value while its count of late jobs rises by only 2.1, so most of its increase comes from jobs that were already late finishing later still. The mean of the iterated greedy sequence rises by 362.6, a factor of 7.25 over its planned 58, against a factor of 1.49 for EDD, because its losses come from jobs that were on time in the plan.

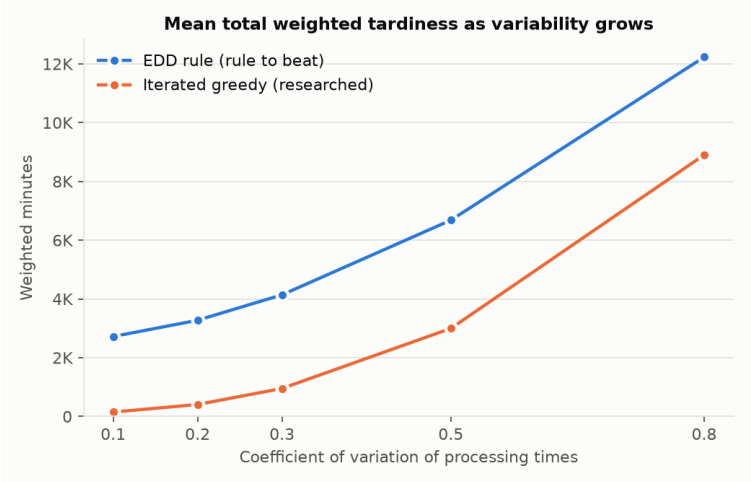


Figure 3: Mean total weighted tardiness of the two fixed sequences for the 45-job instance at five coefficients of variation of the processing times, 30 scenarios each (table 4). The vertical gap between the lines is the absolute reduction, between 2,572.0 and 3,683.7 weighted minutes at every level.

The on-time jobs of the iterated greedy sequence have a median slack of 31 minutes before their due dates, against 28 for EDD, and in each sequence 3 on-time jobs have slack below one standard deviation of their own total processing time at $v = 0.2$, which is $0.2\sqrt{\sum_k p_{jk}^2}$ for job j. The iterated greedy sequence is exposed because it has 41 on-time jobs that a delay can make late, against 11 for EDD. This mechanism is consistent with the observation of Aytug et al. [7] that optimized schedules lose their advantage as variability grows, and the sweep shows that, for weighted tardiness, the loss appears first in the tail of the per-period reduction while its mean stays between 2,572.0 and 3,683.7 weighted minutes. It also extends to this objective the finding of Framinan and Perez-Gonzalez [4] and Juan et al. [5] for the expected makespan, that sequences computed on exact or mean processing times remain effective at low to moderate variability.

7.3 Managerial implications, scope, and extensions

For a planning period like the 45-job instance with processing times varying at a coefficient of variation of 0.2, the searched sequence lowers mean total weighted tardiness by 3,120.5 weighted minutes (standard error 150.1) in the main scenarios and by 2,866.4 (standard error 121.3) in an independent second set, leaves 11.5 jobs late per period on average against 36.1, and lowers mean waiting between stages by 48.0 percent. In exchange, the planner gives up a sequence that can be traced by hand and accepts a result that depends on a seed and takes seconds to compute, 3.22 at 45 jobs and 37.2 at 120. At $v = 0.8$ the mean gain remains 3,329.0 weighted minutes, but the 10th percentile of the per-period gain falls to 851.5 and EDD is better in 1 of 30 scenarios, so a planner who weighs the worst periods most should repeat the comparison at the line's measured variability.

All four instances come from one generator, with processing times of 2 to 20 minutes and median due-date windows of 154 to 158 minutes, and the stress test covers only the 45-job instance, so lines with tighter due dates or more uneven stages may show different margins. The scenarios draw each of the 360 processing times of the 45-job instance independently, so they do not represent a common cause, such as a slow shift or a degraded machine, that lengthens many operations at once. Both sequences

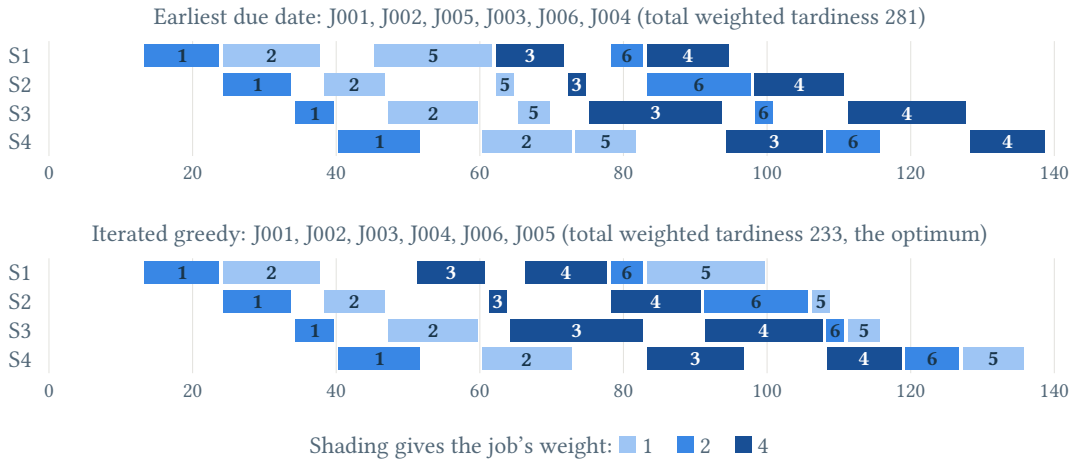


Figure 4: Both schedules of the 6-job instance on stages S1 to S4, with time in minutes on the horizontal axis. Each bar is one operation, labeled with its job number and shaded by the job's weight. Iterated greedy moves the weight-1 job J005 behind the weight-4 jobs J003 and J004.

are predictive and are never revised, which isolates the value of the sequence itself and leaves out rescheduling and reactive dispatching; in the studies reviewed by Aytug et al. [7], dispatching matched or outperformed predictive schedules once variability was high. Optimality is proven only up to 15 jobs, and on the 120-job instance other seeds find better sequences, so the gains reported for the largest instance understate what the search can achieve.

A search that scores each candidate sequence by its mean tardiness over sampled scenarios, in the manner of the simheuristic of Juan et al. [5], would target the variability directly, and the sweep of section 6.3 provides the test. Scenarios with correlated factors, in which one draw lengthens a whole stage or a whole shift, would test whether the absolute gain survives a common cause. Re-sequencing at each release with the processing times already realized would measure how much of the loss under variability a predictive-reactive policy recovers.

8. Conclusions

On exact data, iterated greedy reduced total weighted tardiness relative to EDD by 81.0 to 98.5 percent on the development instances and attained the proven optimum on both instances for which I certified one. Executed unchanged while processing times varied with a coefficient of variation of 0.2, the searched sequence had the lower weighted tardiness in all 30 scenarios and an 88.1 percent lower mean. As the coefficient of variation rose to 0.8, the relative reduction fell to 27.2 percent and the 10th percentile of the per-period reduction to 851.5 weighted minutes, while the mean absolute reduction stayed between 2,572.0 and 3,683.7 weighted minutes.

For a planner who must fix one sequence before processing begins, I found that the mean gain of a searched sequence over EDD held at every level of processing-time variability I tested, and that variability, the one factor I varied, determined how dependably that gain was realized in a single period.

Data availability. The code, the data, every schedule, and every result reported here are available at `https://github.com/mutingma123/OM522Project`, where one command regenerates

every table and figure.

References

[1] Eva Vallada, Rubén Ruiz, and Gerardo Minella. Minimising total tardiness in the m-machine flowshop problem: A review and evaluation of heuristics and metaheuristics. *Computers & Operations Research*, 35(4):1350-1373, 2008. doi: 10.1016/j.cor.2006.08.016. URL https://doi.org/10.1016/j.cor.2006.08.016.

[2] Rubén Ruiz and Thomas Stützle. An Iterated Greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. *European Journal of Operational Research*, 187(3):1143-1159, 2008. doi: 10.1016/j.ejor.2006.07.029. URL https://doi.org/10.1016/j.ejor.2006.07.029.

[3] Kenneth R. Baker and Dominik Altheimer. Heuristic solution methods for the stochastic flow shop problem. *European Journal of Operational Research*, 216:172-177, 2012. doi: 10.1016/j.ejor.2011.07.021. URL https://doi.org/10.1016/j.ejor.2011.07.021.

[4] Jose M. Framinan and Paz Perez-Gonzalez. On heuristic solutions for the stochastic flowshop scheduling problem. *European Journal of Operational Research*, 246:413-420, 2015. doi: 10.1016/j.ejor.2015.05.006. URL https://doi.org/10.1016/j.ejor.2015.05.006.

[5] Angel A. Juan, Barry B. Barrios, Eva Vallada, Daniel Riera, and Josep Jorba. A simheuristic algorithm for solving the permutation flow shop problem with stochastic processing times. *Simulation Modelling Practice and Theory*, 46:101-117, 2014. doi: 10.1016/j.simpat.2014.02.005. URL https://doi.org/10.1016/j.simpat.2014.02.005.

[6] Eliana María González-Neira, Jairo R. Montoya-Torres, and David Barrera. Flow-shop scheduling problem under uncertainties: Review and trends. *International Journal of Industrial Engineering Computations*, 8:399-426, 2017. doi: 10.5267/j.ijiec.2017.2.001. URL https://doi.org/10.5267/j.ijiec.2017.2.001.

[7] Haldun Aytug, Mark A. Lawley, Kenneth McKay, Shantha Mohan, and Reha Uzsoy. Executing production schedules in the face of uncertainties: A review and some future directions. *European Journal of Operational Research*, 161:86-110, 2005. doi: 10.1016/j.ejor.2003.08.027. URL https://doi.org/10.1016/j.ejor.2003.08.027.

[8] HiGHS development team. HiGHS: High performance software for linear optimization, version 1.15.1, 2026. URL https://highs.dev.

[9] Rubén Ruiz and Thomas Stützle. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3):2033-2049, 2007. doi: 10.1016/j.ejor.2005.12.009. URL https://doi.org/10.1016/j.ejor.2005.12.009.

A. Notation

Tables 5 to 7 list every symbol in the report with its meaning, its units, and where it is defined. No symbol carries two meanings.

Table 5: Indices and data.

Symbol	Meaning	Units	Defined in
n	Number of jobs	jobs	section 3.1
m	Number of stages	stages	section 3.1
j	Job index, $j = 1, \ldots, n$		section 3.1
h	Job index in sums and maxima that already involve j		section 3.3
k	Stage index in line order, $k = 1, \ldots, m$		section 3.1
i	Position index within a sequence π		section 3.2
q	Position index in the mixed-integer program		section 3.3
r_j	Release date of job j	minutes	section 3.1
d_j	Due date of job j	minutes	section 3.1
w_j	Weight (priority score) of job j	integer score	section 3.1
p_{jk}	Processing time of job j at stage k	minutes	section 3.1

Table 6: Schedules, measures, and variables of the mixed-integer program.

Symbol	Meaning	Units	Defined in
π, $\pi(i)$	A sequence of all jobs, and the job in its position i		section 3.2
S_{jk}, C_{jk}	Start and completion of job j at stage k	minutes	(3.1)
T_j	Tardiness of job j, $\max\{0, C_{jm} - d_j\}$	minutes	section 3.2
$Z(\pi)$	Total weighted tardiness of π (primary measure)	weighted minutes	(3.2)
$W(\pi)$	Total waiting between stages of π (secondary measure)	minutes	(3.2)
$F(\pi)$	The pair $(Z(\pi), W(\pi))$, compared lexicographically		section 4.2
x_{jq}	1 if job j takes position q, 0 otherwise	binary	section 3.3
B_{qk}, E_{qk}	Start and end of the job in position q at stage k	minutes	section 3.3
U_j	Upper bound on the tardiness of job j in (3.9)	minutes	section 3.3

Table 7: Method and experiment settings.

Symbol	Meaning	Value or units	Defined in
π^{EDD}	Sequence of the earliest due date (EDD) rule		section 4.1
π^*, π', π''	Best sequence found, a rebuilt sequence, and a local search candidate		Algorithm 1
δ	Number of jobs removed in each iteration	4	section 4.2
λ	Temperature factor	0.5	section 4.2
Θ	Temperature, $\lambda \sum_{j,k} p_{jk}/(10\,n\,m)$, compared with increases in Z	minutes	section 4.2
Δ	Increase in Z of a candidate over the current sequence	weighted minutes	section 4.2
L, ℓ	Iteration limit and iteration counter	$L = 5000$	Algorithm 1
u	Uniform random draw on $[0, 1]$ in the acceptance test		Algorithm 1
ω	Scenario index	30 per test	section 5.3
$\varepsilon_{jk\omega}$	Lognormal factor on p_{jk} in scenario ω, mean 1		section 5.3
v	Coefficient of variation of each factor	0.1 to 0.8	section 5.3
σ	Standard deviation of $\ln \varepsilon_{jk\omega}$, $\sigma^2 = \ln(1 + v^2)$		section 5.3